Hengyuan Zhang, Zhihao Zhang, Mingyang Wang, Zunhai Su, Yiwei Wang, Qianli Wang, Shuzhou Yuan, Ercong Nie, Xufeng Duan, Qibo Xue, et al. 2026. Locate, steer, and improve: A practical survey of actionable mechanistic interpretability in large language models. *arXiv preprint arXiv:2601.14004*.

Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023. Adaptive budget allocation for parameter-efficient fine-tuning. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Ruiyi Zhang, Rushi Qiang, Sai Ashish Somayajula, and Pengtao Xie. 2024. Autolora: Automatically tuning matrix ranks in low-rank adaptation based on meta learning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, pages 5048–5060. Association for Computational Linguistics.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2).

Bojia Zi, Xianbiao Qi, Lingzhi Wang, Jianan Wang, Kam-Fai Wong, and Lei Zhang. 2023. Delta-lora: Fine-tuning high-rank parameters with the delta of low-rank matrices. *CoRR*, abs/2309.02411.

A Experiment Details

Datasets. We evaluate our dynamic intervention methods on three datasets: (i) *Psychopathy* (personality tendency classification), (ii) *PowerSeeking* (open-ended generation), and (iii) the top-10 concept subsets from *AxBench* (open-ended generation). To support training and evaluation under our paired-setting and data availability constraints, we construct task-specific train/test splits as follows. For *Psychopathy*, we sample 500 instances for training and 100 for testing. For *PowerSeeking*, we sample 500 instances for training and 200 for testing. For *AxBench*, since its original test set is randomly sampled from an instruction-following corpus and does not provide matched positive/negative answer pairs, we re-split the original 72 instances per concept for each of the top-10 concept subsets into 64 training instances and 8 test instances.

Evaluation and Metrics. For the experiments in §3.3, following Bigelow et al. (2025), we compute **preference** and **utility** log-odds (Eqs. (7) and (8)) for each query q with matched answers (A_p, A_n) on both training and test sets, and vary the intervention scale m to track their changes. For the final performance evaluation in §5.2, we adopt dataset-specific metrics. For *Psychopathy*, following Bigelow et al. (2025), we report classification accuracy (**Acc**). For *PowerSeeking*, following Cao et al. (2024), we use `gpt-4.1-mini` to score generations on the test set on a 0–4 scale. For *AxBench*, following Wu et al. (2025a), we use `gpt-4.1-mini` to evaluate **concept score**, **instruction score**, and **fluency score** on the test set, each on a 0–2 scale; we report the concept score and the **harmonic mean** over the three scores.

Baselines. We evaluate multiple methods under three intervention forms: *local weight updates*, *LoRA*, and *vector* interventions. For each form, we train interventions with either the **SFT** objective or the **RePS** objective (Wu et al., 2025a). For vector interventions, we additionally include a train-free baseline **DiffMean** (Marks and Tegmark, 2023). We also report **Vanilla** results without any steering.

Intervention Setup. We run experiments on `Gemma-2-9B-IT` at layer 20 and on `Qwen-2.5-7B-Instruct` at layer 14, following Bigelow et al. (2025). We consider three intervention forms: *local weight updates*, *LoRA*, and *vector* interventions. For local weight and LoRA, we train intervention parameters on the MLP down-projection matrix; for vector interventions, we apply the intervention directly to the residual stream. For hyperparameters, we largely follow the default settings in Wu et al. (2025a); Xu et al. (2025). We optimize with AdamW and a linear learning-rate scheduler. We also perform reasonable hyperparameter tuning to ensure stable and competitive performance.

Symbol	Description
Unified Analysis Framework	
m	The steering scalar coefficient.
$\mathrm{PrefOdds}(q)$	Preference log-odds ($\mathcal{L}_n - \mathcal{L}_p$). (7)
$\mathrm{UtilOdds}(q)$	Utility log-odds. (8)
$P(u\|q)$	Latent utility probability.
$P(p_\pm\|q)$	Latent preference probability.
$P(\bullet\|h)$	Equivalent to $P(\bullet\|q)$ as the weights remain unchanged.
$\mathcal{L}_p, \mathcal{L}_n$	Cross-entropy losses corresponding to A_p and A_n.
Mechanistic Manifold Model	
$\mathcal{M}_l$	The activation manifold of stably handled inputs at layer l.
$D(m)$	Average validity decay function.
$m_\pm$	Distance from P to P± along the steering line in Fig 3
$L_\pm$	Characteristic scale of decay.
$p_\pm$	Asymptotic decay rate.
Joint Optimization	
$\mathcal{L}_{util}$	Utility loss component.
$\mathcal{L}_{pref}$	Preference loss component.

Table 4: **Notations for Key Concepts.** A summary of the specialized symbols introduced for the unified analysis, mechanistic modeling, and optimization objective.

Results: Unified Dynamics Observation. Figures 2 and 4 show the unified preference–utility dynamics of the `Gemma-2-9B-IT` and `Qwen-2.5-7B-IT` models on the *AxBench* dataset, evaluated over the top-10 concept subsets. And figure 5 shows the unified preference-utility dynamics on *Power-seeking* and *Psychopathy* datasets under different models. We observe that the utility can increase under slight perturbations of m in either the positive or negative direction. In

some cases, this suggests that the origin may not lie exactly on the utility manifold, implying that the utility is not always strictly optimal at $m = 0$.

Results: Performance Comparison. Table 3 compares our method with various baselines under different intervention forms (local weight, LoRA, and vector) on two base models. Across intervention forms, our method remains competitive with strong baselines, and often improves concept metrics while maintaining comparable or higher harmonic scores. The gains are most consistent under LoRA and vector, where our approach typically strengthens concept control relative to both SFT- and RePS-trained variants, and achieves the best or near-best harmonic mean on AxBench in multiple settings. Under full weight updates, we observe smaller but still stable differences, with our method remaining comparable and without an apparent drop in utility. Overall, the results indicate that the proposed optimization transfers across different steering forms and can provide reliable, albeit sometimes incremental, improvements.

B List of Mathematical Symbols

The Table 4 below lists the important symbols used in this paper.

C Derivations and Implementation Details for Log-Odds

This appendix derives the loss-based forms of Eqs. (7)–(8) from the preference–utility independence assumption, and states how we compute the required sequence losses.

C.1 From preference–utility independence to log-odds

For a query q and a polarity pair (A_p, A_n), we assume

$$\begin{aligned} P(A_p \mid q) &= P(u \mid q)\, P(p_p \mid q), \\ P(A_n \mid q) &= P(u \mid q)\, P(p_n \mid q), \end{aligned} \tag{21}$$

with $P(p_p \mid q) + P(p_n \mid q) = 1$.

Preference log-odds. Taking the ratio of (21) cancels $P(u \mid q)$:

$$\frac{P(A_p \mid q)}{P(A_n \mid q)} = \frac{P(p_p \mid q)}{P(p_n \mid q)}. \tag{22}$$

Applying $\log(\cdot)$ gives

$$\mathrm{PrefOdds}(q) \triangleq \log \frac{P(p_p \mid q)}{P(p_n \mid q)} = \log \frac{P(A_p \mid q)}{P(A_n \mid q)}. \tag{23}$$

Using the loss definition $\mathcal{L} \triangleq -\log P(A \mid q)$, we have $P(A \mid q) = e^{-\mathcal{L}}$, and thus

$$\mathrm{PrefOdds}(q) = \log \frac{e^{-\mathcal{L}_p}}{e^{-\mathcal{L}_n}} = \mathcal{L}_n - \mathcal{L}_p, \tag{24}$$

which matches Eq. (7).

Utility probability and log-odds. Summing (21) and using $P(p_p \mid q) + P(p_n \mid q) = 1$ yields

$$\begin{aligned} P(A_p \mid q) + P(A_n \mid q) &= P(u \mid q) \\ &\quad \Big(P(p_p \mid q) + P(p_n \mid q)\Big) \\ &= P(u \mid q). \end{aligned} \tag{25}$$

Therefore,

$$\begin{aligned} \mathrm{UtilOdds}(q) &\triangleq \log \frac{P(u \mid q)}{1 - P(u \mid q)} \\ &= \log \frac{P(A_p \mid q) + P(A_n \mid q)}{1 - P(A_p \mid q) - P(A_n \mid q)}. \end{aligned} \tag{26}$$

Substituting $P(A \mid q) = e^{-\mathcal{L}(A|q)}$ gives the loss form

$$\mathrm{UtilOdds}(q) = \log \frac{e^{-\mathcal{L}_p} + e^{-\mathcal{L}_n}}{1 - e^{-\mathcal{L}_p} - e^{-\mathcal{L}_n}}, \tag{27}$$

which matches Eq. (8). Note that since (A_p, A_n) are only two candidate continuations, we typically have $P(A_p \mid q) + P(A_n \mid q) < 1$.

C.2 Computing sequence losses

Let $A = (y_1, \ldots, y_T)$ be a completion (excluding the query/prompt tokens). We compute the sequence negative log-likelihood (cross-entropy loss) under teacher forcing:

$$\begin{aligned} \mathcal{L}(A \mid q) &\triangleq -\log P(A \mid q) \\ &= -\sum_{t=1}^{T} \log P(y_t \mid q, y_{<t}). \end{aligned} \tag{28}$$

We then set $\mathcal{L}_p \triangleq \mathcal{L}(A_p \mid q)$ and $\mathcal{L}_n \triangleq \mathcal{L}(A_n \mid q)$ and plug them into Eqs. (24) and (27).

Length normalization (optional). When A_p and A_n have different lengths, we optionally use the mean loss $\bar{\mathcal{L}}(A \mid q) \triangleq \mathcal{L}(A \mid q)/T$ in place of $\mathcal{L}(A \mid q)$ to reduce length effects. In that case, the corresponding quantities use $e^{-\bar{\mathcal{L}}}$ instead of $e^{-\mathcal{L}}$.